

Detecting Toxic Comments in Online Discussions:

Preparing the Civil Comments Data

Moaz Allam, Kareem Elhenawy

Department of Computer Science and Engineering, The American University in Cairo

Moaz_Allam@aucegypt.edu, karimelhenawy@aucegypt.edu

CSCE 3602 / DSCI 3415 Class Project, Phase 2: Data Preparation, Cleaning and Feature Generation

ABSTRACT

This report prepares the Civil Comments corpus for the models of the next phase. Phase 1 used an 8-column copy of the data; here we use the full 45-column release. We analysed every column on the training split for type, missing values, distinct values, distribution and relation to the label. The comment text is the only input that both carries signal and exists when a comment is posted. Two columns that look predictive are not legitimate inputs: the number of annotators, whose Spearman correlation with the toxicity score is 0.91 (0.37% of comments seen by 3–5 annotators are toxic, against 39.8% of those seen by 10–99), and the platform's own moderation verdict. The identity columns are missing for 77.59% of comments, and not at random. Cleaning normalised the text and removed 25,351 duplicate training rows and 2,109 training rows whose text also appears in validation or test, leaving 1,777,414, 97,320 and 97,320 comments. Each comment is represented by a 100-dimension Word2Vec embedding of its text and 33 scaled tabular features. No feature correlates with any target beyond $|r| = 0.149$.

KEYWORDS data cleaning, feature engineering, missing values, data leakage, word2vec, toxic comment classification

1 INTRODUCTION

Our project builds a moderation aid: a model that reads a comment and scores it on six toxicity attributes, so that a human moderator can read the worst comments first. Phase 1 surveyed the literature and the available datasets and chose Civil Comments [1]. It also fixed two decisions that this phase builds on: a comment counts as positive for an attribute when at least half of its annotators said so (score ≥ 0.5), and the task is multi-label, since one comment can be both an insult and an identity attack.

This phase turns the raw release into data a model can use. We describe the dataset and why we chose it (Section 2), analyse every column (Section 3), clean and transform the data (Section 4), and list the final features with the size and balance of the cleaned dataset (Section 5). Three rules run through the whole phase. Every statistic and figure comes from a script, and the scripts are submitted with this report. Every relation to the label and every fitted parameter (scalar means, encoder categories, the embedding model) is computed on the training split only, so nothing from validation or test leaks into a decision. And a column becomes a feature only if it would be available at the moment a new comment is posted, because that is when the moderation aid has to score it.

The scripts run in order: `00_download` fetches the data and checks file hashes, `01_load` loads the three files into one table, `02_feature_analysis` produces the analysis in Section 3, `03_clean_preprocess` cleans, encodes and scales the data, `04_word2vec` embeds the text,

`05_correlation_checks` measures the final features, and `06_make_figures` draws every figure.

2 THE DATASET

2.1 Description

Civil Comments was a commenting service used by independent news sites. When it closed at the end of 2017 its archive of public comments was released, and Jigsaw had the comments rated for toxicity; the result was published for the Kaggle competition "Jigsaw Unintended Bias in Toxicity Classification" [3] and is in the public domain (CC0) [4]. The comments were posted between 2015-09-29 and 2017-11-11 on 53 publications.

Each comment was shown to a panel of crowd raters, and each of the seven attributes (toxicity, severe toxicity, obscene, threat, insult, identity attack, sexual explicit) is stored as the fraction of raters who said it applied. A subset of comments was also shown to raters who marked which of 24 identities the comment mentions, in five groups: gender, sexual orientation, religion, race or ethnicity, and disability [2, 3].

The release comes as three files, which we use as three splits: the training file (1,804,874 comments), the public test file as validation (97,320) and the private test file as test (97,320). We found that the files are split by article: none of the 105,380 articles in the training split has a comment in validation or test. Comments in one thread share a topic, vocabulary and quoted text, so a split that ignored articles would show a model half of a conversation in training and the other half at evaluation. We therefore keep the official split.

2.2 Source

Phase 1 used `google/civil_comments` [4], which has only the text and the seven scores. Phase 2 needs every column, which only the original competition files have. Kaggle requires a signed-in account to download them, so we took the three competition files from a public Hugging Face mirror [7].

2.3 Why Civil Comments

Phase 1 compared five public datasets and chose Civil Comments for three reasons [1]. It is the largest, at about two million comments. Its labels are fractions, so a comment eight of ten raters called toxic is distinguishable from one that five of ten did. And it is the only dataset of this size with identity annotations, which our planned bias audit needs [2]. The full release adds a fourth reason that matters for this phase: besides free text it has numeric, nominal, temporal and identifier columns, and real missing values, so every preparation step has something to act on.

3 FEATURE ANALYSIS

Table 1 lists all 45 columns with their type, missing values, number of distinct values and what this phase does with

them. All relations to the label in this section are measured on the 1,804,874 training comments, and the label is either the toxicity score or its binary form $y = (\text{toxicity} \geq 0.5)$. For a numeric column we report the Pearson and Spearman correlations with the score, Pearson's r with the 0/1 label y , and the column's mean in each class. For a nominal column

we report the toxic rate in each category and the information gain $IG = H(y) - H(y | \text{column})$, the entropy-based measure that ID3 uses to choose decision-tree splits. The label's entropy is $H(y) = 0.402$ bits, so IG says how much of that uncertainty a column removes.

Table 1: All 45 columns of the release (missing values and distinct values counted over all 1,999,514 rows).

Column(s)	#	Type	Missing	Distinct values	What Phase 2 does with it
id	1	integer key	0	1,999,514	kept as the row key
comment_text	1	free text	0 (0 empty)	1,971,914	cleaned; encoded as a Word2Vec embedding and 12 statistics
toxicity and 6 subtypes	7	fraction in [0, 1]	0	1,568–3,185	targets, binarised at 0.5; severe_toxicity is not a target
identity columns	24	fraction in [0, 1]	1,551,514 (77.59%)	70–260	bias audit only; not imputed; flag added
identity_annotator_count	1	count	0	164	dropped: duplicates the flag
toxicity_annotator_count	1	count	0	814	not a feature (leaks the label); kept for reference
created_date	1	timestamp (UTC)	0	1,998,662	not a feature: no relation to the label
publication_id	1	nominal	0	53	one-hot, 19 columns + "other"
parent_id	1	identifier	864,806 (43.25%)	786,827	replaced by is_reply
article_id	1	identifier	0	116,943	dropped: an identifier no new comment shares
rating	1	nominal (2 values)	0	2	not a feature: a verdict given after posting
funny, wow, sad, likes, disagree	5	count	0	15–171	dropped: given after posting, no signal

3.1 Labels

Figure 1 shows the distribution of each score over ten bins. The scores are dominated by zeros: 70.07% of training comments have a toxicity score of exactly 0, and between 74.82% and 96.18% have 0 on each subtype. Because panels differ in size, the scores take many distinct values (2,913 for toxicity), not just multiples of one tenth. At the 0.5 threshold, toxicity applies to 144,334 comments (8.00%), insult to 106,534 (5.90%), identity_attack to 13,410, obscene to 9,603, sexual_explicit to 4,686, threat to 4,280, and severe_toxicity to 13. A classifier that always answers "no" would be 92.00% accurate on toxicity and over 99.3% accurate on the rare subtypes, which is why accuracy will not be a headline metric.

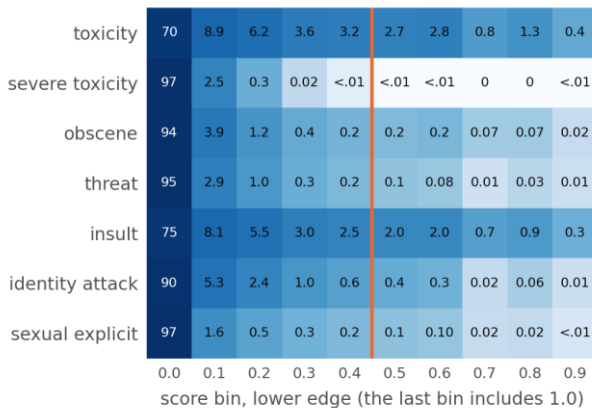


Figure 1: Distribution of the seven label scores over the training split: percentage of comments in each tenth of the score range (colour on a log scale). The orange line is the 0.5 threshold; everything to its right counts as positive.

Figure 2 shows how the seven scores relate. Insult and toxicity move together almost perfectly ($r = 0.93$), and the other subtypes correlate with toxicity at $r = 0.25$ to 0.49 . At the 0.5 threshold, between 87.4% and 100.0% of the comments that carry a subtype are also toxic, yet 15.45% of toxic comments carry no subtype at all. Most comments with

any label carry exactly two (110,980 comments) rather than one (26,477). This is the multi-label structure that calls for one sigmoid output per attribute. The $r = 0.93$ between insult and toxicity is above the 0.85 limit for leakage, which is a second reason the scores are only ever outputs: no score is used as an input to predict another.

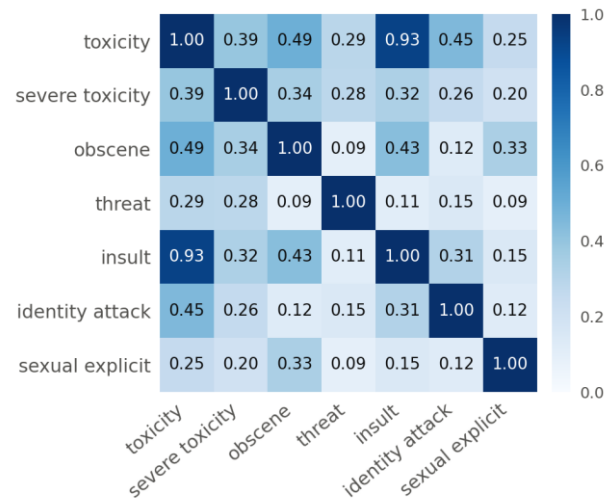


Figure 2: Pearson correlation between the seven label scores (training split). Insult is almost a copy of toxicity; severe_toxicity has 13 positives at 0.5 and is not a modelling target.

3.2 Annotator counts: a leak, not a feature

toxicity_annotator_count ranges from 3 to 4,936 with a median of 4; 1,185,412 training comments (65.7%) had four annotators and 262,216 had ten. Its Spearman correlation with the toxicity score is $\rho = 0.91$, above the 0.85 limit at which a column stands in for the label (Pearson's r is only 0.24, because a few comments had thousands of annotators). It is the only column in the release that crosses the limit. Figure 3 shows why. Comments that were seen by 3–5 annotators are toxic 0.37% of the time; comments seen by 10–99 annotators are toxic 39.8% of the time.

The dataset documentation notes that some comments were shown to far more raters than the usual panel [3]. Whatever the rule, the count is decided by the annotation process after the comment exists, and it depends on how toxic the comment looked. A new comment arriving at the moderation aid has no annotator count. A model trained with it would learn that "many annotators means toxic" and would have nothing to apply that rule to in use, which is leakage. It is therefore not a feature. The cleaned data keeps it as `n_annotators` for reference only. `identity_annotator_count` is 0 for 77.55% of training comments and is non-zero exactly when the identity columns are present (checked on every row), so it adds nothing beyond a flag and is dropped.

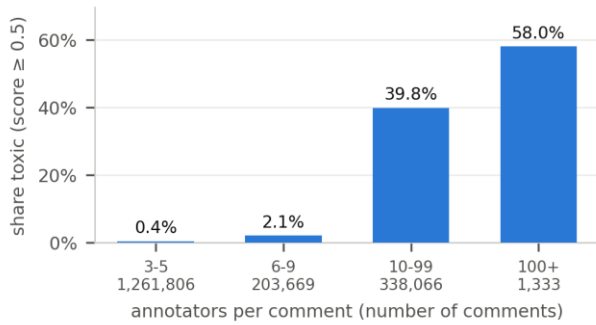


Figure 3: Share of training comments that are toxic, by how many annotators rated them. The count is set by the annotation process, not by the comment, and is unavailable for a new comment.

3.3 Comment text

No comment is missing or empty. Across all splits there are 1,971,914 distinct texts in 1,999,514 rows. Training comments have a median of 202 characters and 35 words, a mean of 297 characters, and a maximum of 1,906; the length distribution is right-skewed (skew 1.23) with a long tail of essays. Figure 4 compares the word-count distributions of toxic and non-toxic comments. They overlap almost completely; toxic comments are slightly shorter on average (278 against 299 characters).

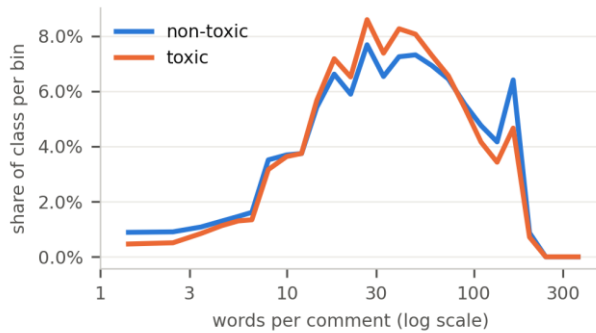


Figure 4: Distribution of words per comment for toxic and non-toxic training comments (each curve sums to 100%). Length alone barely separates the classes.

Reading the raw text turned up several things to clean. Some comments use Windows line breaks, contain invisible control characters, or use typographic variants such as "...". and styled letters that Unicode normalisation turns into plain text; 6 contain HTML entities such as "&"; 839 training comments contain something shaped like an HTML tag, but on inspection these are people's own markup such as "<sarcasm>" or "<i>", so they are kept as text. 10,180 distinct texts occur more than once in the training split,

covering 34,231 rows of which 24,051 are redundant copies; the most repeated are short replies such as "Well said." (184 times) and "No." (160). For 596 of these texts the copies disagree on the label. Finally, 1,522 validation and 1,554 test comments have an exact copy in the training split.

To see how much can be read from the style of a comment without its words, we counted 15 statistics per comment: length in characters, letters and words, mean word length, upper-case letters and their ratio to all letters, fully capitalised words of three or more letters ("shouting"), exclamation and question marks, runs of repeated "!" or "?", words masked with asterisks, links, digits, line breaks and non-ASCII characters. Figure 5 gives their Pearson correlation with the 0/1 label `y`. Every one is weak. The strongest are masked words (+0.050), digits (−0.044) and exclamation marks (+0.042); digits and links lean non-toxic, because comments that cite figures and sources tend to be arguments rather than attacks. The conclusion is that toxicity lives in the words, not the typography, which is why the text is encoded word by word with Word2Vec (Section 4.7). Its dimensions correlate with the labels several times more strongly than any style statistic (Section 5.2).

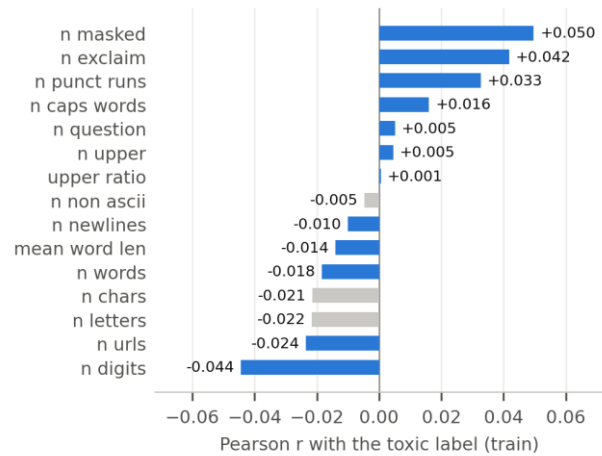


Figure 5: Pearson correlation of each candidate text statistic with the toxic label (training split). Blue: kept; grey: dropped (Section 4.6). The largest $|r|$ is 0.050.

3.4 Identity columns

The 24 identity columns are missing together: a comment has all 24 or none (0 partially missing rows), and they are present for 405,130 training comments (22.45%). The missingness is not random: the annotated subset is more toxic than the corpus as a whole (11.36% against 8.00%). Inside it, a comment that mentions any identity is toxic 16.4% of the time and one that mentions none 7.7%; the largest correlation of a single identity with the score is $r = 0.194$. The identity columns are annotations of the text made after the fact, not something a new comment comes with, so they are never model inputs; they are kept for the bias audit planned in Phase 1 [1, 2].

3.5 Metadata

rating records the platform's moderation outcome: 1,684,758 training comments were approved and 120,116 rejected. Rejected comments are toxic 26.0% of the time against 6.7% for approved ones (information gain 0.0155 bits, 3.86% of $H(y)$, the largest of any metadata column). The verdict was given by people judging the same comment after it was written, so it is a second label rather than an input, and it is not a feature.

publication_id has 53 values. The ten largest publications hold 97.9% of training comments, while 34 publications have fewer than 1,000 comments each (6,562 comments in total). Every publication in validation and test also appears in training. Figure 6 shows that the toxic rate differs from site to site, from 1.46% to 16.31% among publications with at least 1,000 comments (information gain 0.0031 bits, 0.78% of $H(y)$). The publication is known when a comment is posted, so it is kept as a feature.



Figure 6: Share of toxic comments for each publication with at least 1,000 training comments. The orange line is the rate over all training comments.

created_date spans 2015-09-29 to 2017-11-11, and all three splits cover the same range, so the split is not by time. Only 651 training comments are from 2015. The date carries almost nothing about the label. Split into year, month, hour and day of the week, no part removes more than 0.08% of the label's entropy (information gain 0.000004, 0.0003, 0.0002 and 0.0001 bits), and the toxic rate is 7.94% in 2016 and 8.02% in 2017. The year would also never generalise: a model in use after 2017 would only see years it was not trained on.

parent_id is missing for 43.14% of training comments, but these values are not lost: a top-level comment has no parent. The useful information is whether the comment is a reply. Top-level comments are toxic 8.62% of the time and replies 7.52% (information gain 0.0003 bits, 0.07% of $H(y)$). The parent comment itself is in the dataset for 98.0% of replies, which a later phase could use as context.

article_id identifies 105,380 training articles with a median of 4 comments each (maximum 1,377). Longer threads are slightly more toxic (Spearman $\rho = 0.055$ between thread size and score), but the size of a thread is only known after it has grown, and no article is shared with validation or test. **Reader reactions** (funny, wow, sad, likes, disagree) are zero for 41.1% to 96.2% of comments, are heavily right-skewed (skew 6.9 to 11.8), arrive after posting, and barely relate to the label ($|\rho| \leq 0.044$).

3.6 Summary

Table 2 puts the candidate inputs side by side. The two strongest columns, the annotator count and the moderation verdict, are both produced after a comment is written and would leak the label; the annotator count is also above the 0.85 correlation limit. Of the columns available at posting time, the publication carries a little signal and the style statistics carry little. The reply flag and the date are both very weak (under 0.1% of $H(y)$ each). We keep the reply flag because it is a structural fact about the comment that costs one binary column, and drop the date because none of its parts is stronger and the year cannot generalise. Almost all of the usable information is in the words, which Section 4.7 encodes.

Table 2: Relation of each candidate input to the toxic label (training split) and whether it exists when a comment is posted. IG is information gain in bits, out of $H(y) = 0.402$.

Candidate	Relation to y	At posting?	Feature?
toxicity_annotator_count	$\rho 0.913 (\geq 0.85)$	no	no
rating	IG 0.0155	no	no
identity_columns	$r \leq 0.194$	no	no
publication_id	IG 0.0031	yes	yes
thread_size(article_id)	$\rho 0.055$	no	no
reactions	$ \rho \leq 0.044$	no	no
is_reply(parent_id)	IG 0.0003	yes	yes
created_date	IG ≤ 0.0003	yes	no
text_statistics	$ r \leq 0.050$	yes	12 of 15
words(Word2Vec)	$ r \leq 0.149$	yes	yes

4 CLEANING AND PRE-PROCESSING

Cleaning happens in a fixed order. Row-level text cleaning is applied identically to every split, since it needs no fitted parameter. Row filtering is applied to the training split only, so validation and test remain the benchmark rows and results stay comparable with published work on the same split. Every transformation that learns something from the data (feature selection, percentiles, means and standard deviations, one-hot categories and the Word2Vec model) is fitted on the cleaned training split and then applied unchanged to validation and test. Table 3 tracks the number of rows through each step.

Table 3: Rows after each cleaning step.

Step	Train	Validation	Test
Raw release	1,804,874	97,320	97,320
A. Text cleaned, empty removed	1,804,874	97,320	97,320
B1. Duplicates collapsed	1,779,523	97,320	97,320
B2. Overlap with val/test removed	1,777,414	97,320	97,320

4.1 Text cleaning

Each comment is cleaned into two versions. The first, **text**, keeps case and punctuation for models that read raw text, such as the transformer planned for Phase 3. It is produced by converting Windows line endings, applying Unicode NFKC normalisation, deleting control and zero-width characters, collapsing runs of spaces and tabs, allowing at most one blank line, trimming, and unescaping HTML entities. These steps changed 838,146 comments (41.9%), mostly by collapsing the double space after a full stop, and removed 2,549,308 characters in total. No comment became empty.

The second, **text_bow**, is the input to Word2Vec. It lower-cases the text; replaces links with the token "xxurl", words masked with asterisks with "xxmasked" and numbers with "xxnum"; expands contractions; and replaces every remaining character that is not a letter with a space. The placeholders keep the fact that a link, a masked swear word or a number was present without creating a separate word for every variant. Contractions are expanded before apostrophes are removed because removing them first turns "he'll" into "hell" and "we'll" into "well", inventing profanity. We do not remove stop words: "you", "your" and "they" are the words that aim an insult at someone. We do not stem either: Word2Vec already places spelling variants of a word next to each other, as the neighbour check in Section 4.7 shows. 88 comments consist only of punctuation or emoji and have an

empty `text_bow`; they keep their row and receive the zero embedding.

4.2 Duplicates and overlap between splits

After text cleaning, 36,567 training rows belong to 11,216 groups of identical texts. Each group is collapsed to the copy with the most annotators, whose label is the most reliable. This removes 25,351 rows, including the copies in the 684 groups whose copies disagree on the label. Then every training row whose text also occurs in validation or test is removed (2,109 rows), since a model could otherwise memorise an evaluation answer. Validation and test are not deduplicated, so they remain the official benchmark.

4.3 What was not removed

Only 53 training comments had fewer than four annotators, and their scores are still exact fractions of their votes, so they stay. Very short comments ("No.", "BS") are real comments and some are toxic, so they stay too. There are no runaway lengths to cap: the longest comment has 1,906 characters and the 99th percentile is 999. We did not resample the data to balance the classes. Copies of rare comments would leak across cross-validation folds, and the test split has to keep the real balance.

4.4 Missing values

Only two columns have missing values (Table 1). For `parent_id`, the missing value means "top-level comment", so the column is replaced by a binary `is_reply` with no missing values. The identity columns are deliberately not imputed. The usual fills (zero, the mean, the most common value) would all claim something about 77.59% of comments that nobody checked: filling with 0 would assert that no identity is mentioned. Because the missingness is not at random (Section 3.4), even the mean of the annotated subset would be biased. The columns are kept as they are, with a flag `identity_annotated`, and the bias audit will use only annotated comments. No other column has a missing value in any split.

4.5 Labels

The seven scores are kept, and a binary target `y_attribute = (score ≥ 0.5)` is added for each. Six are modelling targets. `severe_toxicity` is not: with 13 positives in 1,804,874 training comments there is nothing to learn, and its score is kept only for completeness. Keeping the scores makes the threshold sensitivity check from Phase 1 possible later at 0.4 and 0.6.

4.6 Encoding, transformation and scaling

Text statistics. Of the 15 candidates, a statistic is dropped if it adds nothing new ($|r| > 0.9$ with a statistic already kept) or if it has no relation to the label ($|p| < 0.01$ and $|r| < 0.01$ with y). `n_non_ascii` is unrelated to the label ($\rho = -0.002$, $r = -0.005$); `n_chars` is redundant with `n_words` ($r = 0.992$); `n_letters` is redundant with `n_words` ($r = 0.990$). That leaves 12. Figure 7 shows why they need transforming: the counts are piled up at zero with long right tails (skew 39.3 for exclamation marks and 1.23 for words). Each count is replaced by $\log(1 + x)$, which pulls the tail in (skew 3.37 and -0.30 afterwards). The two ratios, mean word length and the upper-case ratio, are capped at their 1st and 99th training percentiles instead, since comments made only of a link have mean word lengths above 40. Every statistic is then standardised to $z = (x - \text{mean}) / \text{std}$ with the training mean and standard deviation.

Scaling matters because the models planned for Phase 3 are sensitive to it. In k-nearest neighbours, word counts in

the hundreds would swamp a ratio between 0 and 1 in every distance. In logistic regression, the SVM and the neural network, features on different scales slow gradient descent and make regularisation punish features unevenly. We use standardisation rather than min-max scaling because one extreme comment would set the maximum and squash every other value into a narrow band.

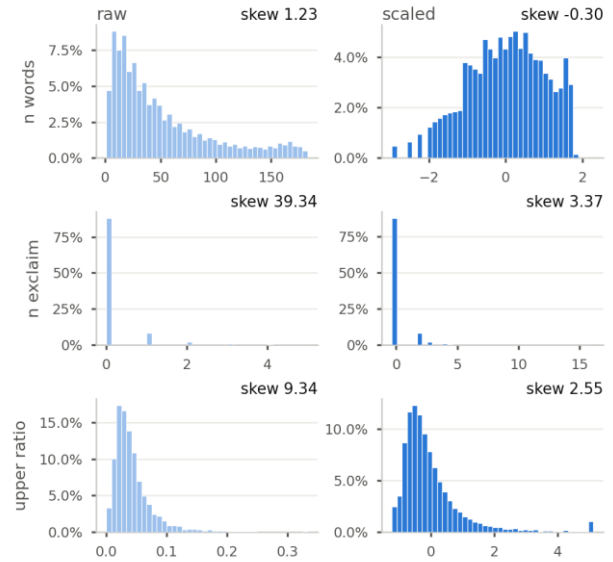


Figure 7: Three text statistics before (left, raw) and after (right, scaled) their transform and standardisation, training split. Counts get $\log(1 + x)$; ratios are capped at the 1st/99th percentiles. Each panel's skew is printed above it.

Publication. `publication_id` is nominal: publication 54 is not "more" than publication 6. Label encoding would invent that order for linear models and KNN, and frequency encoding would merge publications of similar size. We therefore one-hot encode it. The 19 publications with at least 1,000 training comments get a column each; the 34 smaller ones, and any publication never seen in training, share one "other" column, so no column is almost empty and an unknown site still has a valid encoding. That gives 20 binary columns. **Reply.** `is_reply` is already 0/1 and is not rescaled.

4.7 Text representation

The comment text has to be either one-hot encoded or embedded. A one-hot encoding would need a column for every word in a vocabulary of tens of thousands, almost all zero for any one comment, and would treat "idiot" and "moron" as unrelated columns. We therefore embed the text with Word2Vec, which places words used in similar contexts close together. We trained a skip-gram model [5, 6] on the training comments only, with 100 dimensions, a window of 5 words, words seen at least 5 times, 5 negative samples and 5 epochs. We chose skip-gram over CBOW because skip-gram learns semantic relationships between words, while CBOW mostly learns syntactic, positional similarity, and toxicity is a matter of meaning: "idiot" and "moron" should be close because they mean the same thing. Skip-gram is slower to train and needs more data, which a training corpus of this size provides. The model learned 82,724 words from 93,291,771 training tokens, and 99.7% of validation tokens are in its vocabulary. A quick check confirms that it learned meaning: the nearest neighbours of "idiot" are "imbecile", "moron", "asshat", "ignoramus", and those of "thanks" are "thank", "thx", "mahalo", "thanx". A

comment's embedding is the mean of its words' vectors (the zero vector for the 828 training comments with no known word). Each of the 100 dimensions is then standardised with the training mean and standard deviation, like the other numeric features.

5 FINAL FEATURES AND DATASET

Table 4 lists the final feature set and Table 5 the size and balance of the cleaned data. Before accepting the features we ran two checks on the training split.

Table 4: The final feature set. All parameters are fitted on the cleaned training split.

Block	Columns	Content	Encoding and scaling
Words (Word2Vec)	100	mean skip-gram vector of the comment's words	z-score per dimension
Text statistics	12	n words, mean word len, upper ratio, n caps words, n exclaim, n question, n punct runs, n masked, n urls, n digits, n newlines, n upper	counts: $\log(1 + x)$; ratios: capped at p1/p99; then z-score
Reply flag	1	is_reply (from parent_id)	binary 0/1
Publication	20	19 publications + "other"	one-hot; rare or unseen publications go to "other"
Targets	6	y_toxicity, y_insult, y_identity_attack, y_obscene, y_threat, y_sexual_explicit	score ≥ 0.5 ; the seven scores are kept
Kept, not inputs	—	n_annotators, 24 identity scores + identity_annotated (bias audit), text, text_bow, publication_id, created_date, rating	unchanged; never given to a model as input
Removed	—	identity_annotator_count, parent_id, article_id, five reaction counts, n_chars, n_letters, n_non_ascii	reasons in Sections 3 and 4.6

5.1 Feature-to-feature correlation

Figure 8 is the correlation matrix of the 13 scaled tabular features. The strongest pair is n words with n upper ($r = 0.75$), below the 0.9 redundancy cut. Every other pair is weaker, so each statistic adds something its neighbours do not. The publication columns correlate with each other only negatively, as one-hot columns of a single variable must: a comment belongs to one publication, so a 1 in one column means 0 in the rest. The strongest pair is publications 21 and 54 ($r = -0.37$), the two largest sites. Their relation to the other features is weak (largest $|r| = 0.12$).

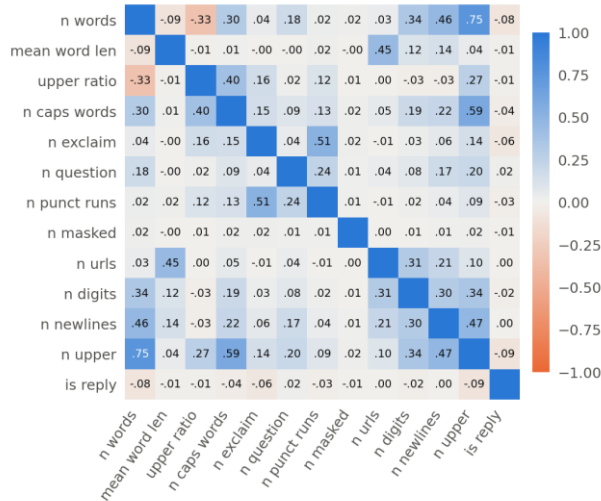


Figure 8: Pearson correlation between the scaled tabular features (training split). Blue is positive, orange negative.

Figure 9 extends the check to all 133 final features by adding the 100 Word2Vec dimensions. Of the 8,778 pairs of features, none exceeds 0.9 and only 3 exceed 0.5, all of them pairs of text statistics already shown above. The strongest pair of Word2Vec dimensions has $r = -0.44$ (dimensions 22 and 64), and the strongest relation between a Word2Vec dimension and a tabular feature is $r = -0.31$ (dimension 40 with n words). No feature is a copy of another, and the embedding carries information that the tabular features do not, so no feature is removed for redundancy.

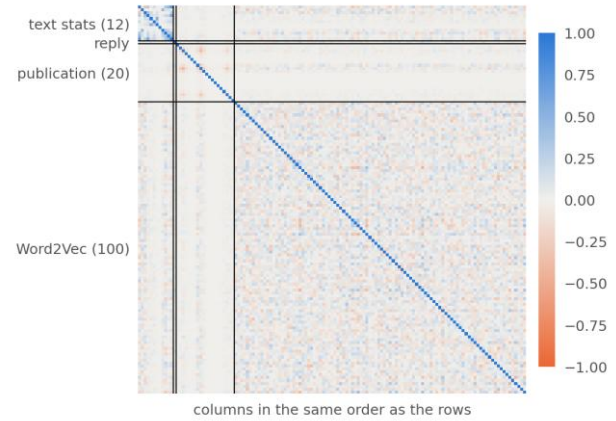


Figure 9: Pearson correlation between all 133 final features (training split): 12 text statistics, the reply flag, 20 publication columns and 100 Word2Vec dimensions. Blue is positive, orange negative; the diagonal is each feature with itself.

5.2 Correlation with the labels and the leakage check

A feature that correlates with a target at $|r| \geq 0.85$ would be standing in for the label rather than helping to predict it, so any such feature has to be dropped. `05_correlation_checks` computes Pearson's r between every final feature and each of the six targets on the training split, and stops with an error if any value reaches 0.85. Figure 10 shows the values for the tabular features. The largest absolute correlations are 0.100 for the tabular features (n masked with obscene), and 0.149 for the 100 Word2Vec dimensions (dimension 57 with toxicity). All are far below 0.85, so no feature is removed by this rule. The only pair in the data that does exceed it is two labels, insult and toxicity ($r = 0.93$), and labels are never inputs. Before cleaning, the one input column above the limit was toxicity_annotator_count ($\rho = 0.91$), which Section 3.2 removed.

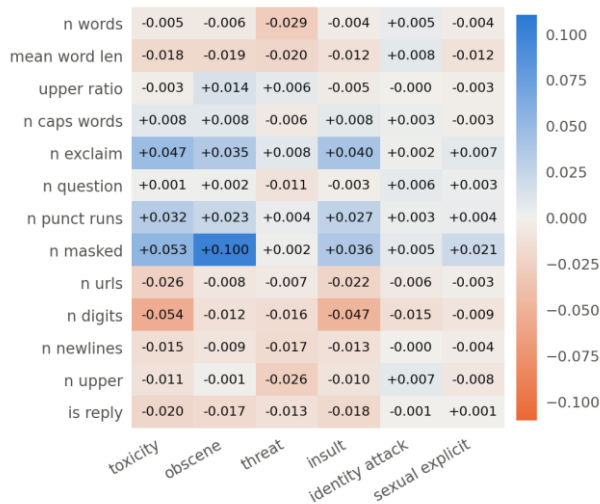


Figure 10: Pearson correlation between each scaled tabular feature and each of the six targets (training split). The leakage limit is $|r| = 0.85$; the largest value is 0.100.

5.3 The cleaned dataset

Table 5: The cleaned dataset: comments and positives per target at 0.5.

	Train	Validation	Test
comments	1,777,414	97,320	97,320
toxicity	142,239 (8.00%)	7,671 (7.88%)	7,777 (7.99%)
insult	104,834 (5.90%)	5,769 (5.93%)	5,776 (5.94%)
identity attack	13,265 (0.75%)	664 (0.68%)	687 (0.71%)
obscene	9,497 (0.53%)	525 (0.54%)	543 (0.56%)
sexual explicit	4,647 (0.26%)	201 (0.21%)	240 (0.25%)
threat	4,227 (0.24%)	224 (0.23%)	221 (0.23%)
severe toxicity	13 (0.0007%)	0 (0.00%)	0 (0.00%)
identity annotated	400,359	21,293	21,577

The cleaned data holds 1,972,054 comments: 1,777,414 for training, 97,320 for validation and 97,320 for testing. Cleaning did not change the class balance, which stays at 8.00% toxic in training and 7.88% and 7.99% in validation and test. The mean comment is 298 characters (median 203). Each split is stored as a Parquet file of 80 columns (501 MB for training) with its row-aligned Word2Vec matrix. Every fitted parameter (clip limits, means, standard deviations, publication categories and the word vectors) is saved next to them, so the same pipeline can be applied to a new comment.

Cleaned dataset:

<https://drive.google.com/drive/folders/1zVfZamzpx0P-tyvNQ8fbEE2nyBqw1Dcy?usp=sharing>

6 CONCLUSION

The full Civil Comments release has 45 columns, but the analysis shows that one of them, the comment text, does nearly all the work. The columns that looked most predictive were produced after posting: the annotator count, the moderation verdict, reader reactions and thread size. Using them would have meant training on the answer. The date carries no signal, and the identity columns are missing for most comments and are kept for auditing rather than prediction. Cleaning removed duplicate and leaking training rows without touching the benchmark splits. It left 1,777,414 training comments represented by a Word2Vec embedding and 33 scaled tabular features, none of which approaches the

leakage limit. Phase 3 can start training immediately: the features are encoded and scaled, and the bias audit has its identity columns.

AI USAGE DISCLOSURE

This report was completed with assistance from Claude (Anthropic). The AI was used to write analysis, and feature-generation scripts, to derive the statistics and draw the figures from their outputs. All numbers in the report are read from the outputs of the submitted scripts, the decisions were reviewed by the authors, and the authors are accountable for the accuracy of the content.

REFERENCES

- [1] Moaz Allam and Kareem Elhenawy. 2026. Detecting Toxic Comments in Online Discussions: A Literature Review and Dataset Survey. CSCE 3602 / DSCI 3415 class project, Phase 1 report. The American University in Cairo.
- [2] Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2019. Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. In Companion Proceedings of The 2019 World Wide Web Conference. arXiv:1903.04561
- [3] cjadams, Daniel Borkan, inversion, Jeffrey Sorensen, Lucas Dixon, Lucy Vasserman, and nithum. 2019. Jigsaw Unintended Bias in Toxicity Classification. Kaggle. kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification
- [4] Google. civil_comments. Hugging Face Datasets, CC0-1.0. huggingface.co/datasets/google/civil_comments
- [5] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781
- [6] Radim Řehůřek and Petr Sojka. 2010. Software Framework for Topic Modelling with Large Corpora. In Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks. ELRA, 45-50.
- [7] shuttie. jigsaw-unintended-bias (mirror of the Kaggle competition files). Hugging Face Datasets. huggingface.co/datasets/shuttie/jigsaw-unintended-bias